

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

2026.05.29

데이터베이스 기본 개념

데이터베이스를 엑셀에 비유하면 구조를 이해하기 쉽다. 엑셀 파일 안에 여러 시트가 있고, 각 시트에 표가 들어가는 것처럼 데이터베이스도 큰 저장 공간 안에 여러 테이블을 둔다.

DataBase 또는 Schemas

- 학생관리 >> 파일
- 학생명단 테이블 >> 시트
- 출석현황 테이블
- 기타 관리 테이블

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 MySQL Workbench에서 보이는 `SCHEMAS`는 데이터베이스 목록이라고 이해하면 된다. 하나의 스키마 안에 여러 테이블을 만들 수 있다.

핵심 데이터베이스는 데이터를 저장하는 공간이고, 테이블은 실제 데이터를 표 형태로 담는 단위이다.

DBMS

DBMS는 Database Management System의 약자로, 데이터베이스를 만들고 관리하는 프로그램이다.

MySQL	수업에서 사용한 관계형 데이터베이스 관리 시스템
Oracle	기업 환경에서 많이 쓰이는 관계형 데이터베이스 관리 시스템

정리 데이터베이스가 데이터 저장 공간이라면, DBMS는 그 저장 공간을 만들고 조회하고 수정하게 해주는 관리 프로그램이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

관계형 데이터베이스 RDB

관계형 데이터베이스는 데이터를 표 형태의 테이블로 나누고, 테이블 사이를 키로 연결해서 관리하는 방식이다.

스키마 정의	데이터베이스와 테이블 구조를 미리 설계
테이블	데이터를 저장하는 표
컬럼	저장 항목 또는 필드
레코드(row)	실제 데이터 한 줄
고유키(PK)	각 레코드를 구분하는 기본키
참고키(FK)	다른 테이블의 기본키를 참고하는 외래키

핵심 관계형 데이터베이스는 여러 표를 따로 만들고, 필요한 관계는 PK와 FK로 연결한다.

정규화

정규화는 중복되는 데이터를 줄이고, 핵심키끼리 연결되도록 테이블을 나누는 과정이다.

주석 예를 들어 학생 정보와 출석 정보를 한 표에 계속 반복해서 저장하면 같은 이름, 학번, 학과 정보가 여러 번 중복될 수 있다. 이때 학생 테이블과 출석 테이블을 나누고, 학생을 구분하는 키로 연결하면 중복을 줄일 수 있다.

정리 정규화는 데이터를 깔끔하게 나누고, 중복을 줄이고, 키를 기준으로 다시 연결하는 설계 방법이다.

SQL 명령어 분류

DDL	데이터베이스 구조를 정의하는 명령. 예: CREATE, ALTER, DROP
DML	데이터를 조회/추가/수정/삭제하는 명령. 예: SELECT, INSERT, UPDATE, DELETE
DCL	권한을 제어하는 명령. 예: GRANT, REVOKE

확인 테이블을 만드는 것은 DDL, 테이블 안의 데이터를 다루는 것은 DML, 사용자 권한을 주고 빼는 것은 DCL로 구분한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

NoSQL

NoSQL은 관계형 테이블 구조만 사용하는 방식이 아니라, 목적에 따라 더 유연한 형태로 데이터를 저장하는 방식이다.

Key-Value	key와 value 쌍으로 저장
Document	JSON 문서처럼 필드 구조를 가진 문서 단위로 저장

주의 NoSQL은 SQL을 전혀 안 쓴다는 뜻보다, 전통적인 관계형 테이블 방식만 고정해서 쓰지 않는 데이터베이스 계열로 이해하면 된다.

MySQL Workbench 스키마 생성

MySQL Workbench에서 왼쪽 `SCHEMAS` 영역을 이용해 새 스키마를 만들었다.

```
SCHEMAS 우클릭
-> Create Schema
-> Schema Name: my_community
-> Charset/Collation: utf8mb4 / utf8mb4_general_ci
-> Apply
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `my\_community`는 커뮤니티 서비스용 데이터베이스 이름으로 만든 스키마이다.

주석 `utf8mb4`는 한글과 이모지까지 저장할 수 있는 문자셋이다. 수업 메모에는 `utf8mb4\_general\_`처럼 적었지만 Workbench에서는 보통 `utf8mb4`와 `utf8mb4\_general\_ci`를 선택한다.

확인 스키마 생성 후 왼쪽 `SCHEMAS` 목록에 `my\_community`가 보이면 생성된 것이다.

MySQL Workbench 테이블 생성

`my\_community` 스키마 안에서 `Tables`를 선택하고 새 테이블을 만들었다.

```
my_community
-> Tables
-> Create Table
-> Table Name: user
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

수업 캡처 기준으로 `user` 테이블에 아래 컬럼을 작성했다.

```
user_id VARCHAR(50) PRIMARY KEY NOT NULL
age INT UNSIGNED
nickname VARCHAR(200)
email VARCHAR(200)
password VARCHAR(300)
```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

```
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

- 주석

``user_id``는 사용자를 구분하는 값이므로 PK와 NN을 체크했다. PK는 Primary Key, NN은 Not Null을 의미한다.
- 주석

``age``는 나이를 저장하는 숫자 컬럼이고, 음수가 될 필요가 없으므로 UN(Unsigned)을 체크했다.
- 주석

``created_at``은 가입 또는 생성 시각을 저장하는 컬럼이다. 기본값을 ``CURRENT_TIMESTAMP``로 두면 데이터가 생성될 때 현재 시간이 자동으로 들어간다.
- 결과

하단 Action Output에 ``Changes applied``가 표시되면 Workbench에서 테이블 생성 또는 수정 적용이 완료된 것이다.
- 정리

오늘 실습은 ``my_community`` 스키마를 만들고, 그 안에 사용자 정보를 저장할 ``user`` 테이블 구조를 설계하는 흐름까지 진행했다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

SQL 원문과 실행 의미

수업에서 MySQL Workbench로 만든 `user` 테이블은 SQL로 보면 아래와 같은 `CREATE TABLE` 명령으로 표현된다.

```
CREATE TABLE `my_community`.`user` (  
  `user_id` VARCHAR(50) NOT NULL,  
  `age` INT UNSIGNED NULL,  
  `nickname` VARCHAR(200) NULL,  
  `email` VARCHAR(200) NULL,  
  `password` VARCHAR(300) NULL,  
  `created_at` DATETIME NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`user_id`)); -- 테이블생성 명령
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

- 주석** `CREATE TABLE`은 새 테이블을 만드는 DDL 명령이다. 여기서는 `my\_community` 스키마 안에 `user` 테이블을 만든다.
- 주석** `user\_id VARCHAR(50) NOT NULL`은 사용자 아이디를 최대 50글자 문자열로 저장하고, 비워둘 수 없게 설정한 컬럼이다.
- 주석** `age INT UNSIGNED NULL`은 나이를 정수로 저장하고, 음수는 허용하지 않으며, 값이 없어도 되는 컬럼이다.
- 주석** `created\_at DATETIME NULL DEFAULT CURRENT\_TIMESTAMP`는 생성 시간을 저장하는 컬럼이다. 값을 직접 넣지 않으면 현재 시간이 기본값으로 들어간다.
- 주석** `PRIMARY KEY (user\_id)`는 `user\_id`를 기본키로 지정한다. 기본키는 테이블에서 각 행을 구분하는 대표 값이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

데이터베이스와 테이블 확인 명령

```
show databases; -- 데이터베이스 목록 조회
use my_community; -- 사용할 데이터베이스 스키마 선택
show tables; -- 선택된 스키마의 테이블 목록 조회
select * from user; -- user 테이블의 전체 데이터 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

show databases	현재 MySQL 서버에 있는 데이터베이스/스키마 목록을 확인한다.
use my_community	앞으로 실행할 SQL 명령의 대상 스키마를 `my_community`로 선택한다.
show tables	현재 선택된 스키마 안에 있는 테이블 목록을 확인한다.
select * from user	`user` 테이블의 모든 컬럼과 모든 행을 조회한다.

확인 `select \*`에서 `\*`는 모든 컬럼을 뜻한다. 실무에서는 필요한 컬럼만 골라 조회하는 방식도 많이 사용한다.

데이터 입력

`user` 테이블에 테스트 데이터를 한 줄 추가했다.

```
insert into user (user_id, age, nickname) values ('abc', 28, '테스트');
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `insert into`는 테이블에 새 데이터를 추가하는 DML 명령이다.

주석 `(user\_id, age, nickname)`은 값을 넣을 컬럼 목록이고, `values ('abc', 28, '테스트')`는 실제로 저장할 값이다.

결과 위 명령을 실행하면 `user\_id`가 `abc`, `age`가 `28`, `nickname`이 `테스트`인 사용자 데이터가 추가된다.

주의 `user\_id`는 기본키이므로 같은 `user\_id` 값을 중복해서 넣으면 오류가 날 수 있다.

정리 오늘 SQL 흐름은 `테이블 생성 -> 스키마 선택 -> 테이블 확인 -> 데이터 조회 -> 데이터 입력` 순서로 보면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

sample.sql 파일로 테이블 생성

`C:\Users\wai\Downloads\sample.sql` 파일을 MySQL Workbench에서 실행해서 커뮤니티 예제용 테이블을 만들었다.

핵심

SQL 파일은 여러 SQL 명령을 한 번에 저장해 둔 스크립트 파일이다. Workbench에서 실행하면 테이블 생성, 데이터 입력, 조회 연습 쿼리를 순서대로 실행할 수 있다.

```
-- 기존 테이블 삭제
DROP TABLE IF EXISTS comments;
DROP TABLE IF EXISTS posts;
DROP TABLE IF EXISTS users;

-- 유저 테이블
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  nickname VARCHAR(30) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`DROP TABLE IF EXISTS`는 같은 이름의 테이블이 이미 있으면 삭제하는 명령이다. 실습을 다시 실행할 때 기존 테이블과 충돌하지 않게 한다.

주석

`comments -> posts -> users` 순서로 삭제하는 이유는 외래키 관계 때문이다. 자식 테이블을 먼저 지워야 부모 테이블 삭제 오류를 줄일 수 있다.

주석

`AUTO\_INCREMENT`는 새 행이 추가될 때 숫자 기본키가 자동으로 증가하게 하는 설정이다.

주석

`UNIQUE`는 같은 이메일이 중복 저장되지 않도록 막는 제약조건이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

게시물/댓글 테이블 생성

`posts` 테이블은 게시글을 저장하고, `comments` 테이블은 댓글을 저장한다. 두 테이블 모두 사용자 테이블의 `user\_id`를 외래키로 참고한다.

```
CREATE TABLE posts (
  post_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  title VARCHAR(200) NOT NULL,
  content TEXT NOT NULL,
  view_count INT DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE comments (
  comment_id INT AUTO_INCREMENT PRIMARY KEY,
  post_id INT NOT NULL,
  user_id INT NOT NULL,
  content VARCHAR(500) NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (post_id) REFERENCES posts(post_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `FOREIGN KEY (user\_id) REFERENCES users(user\_id)`는 게시글/댓글 작성자가 `users` 테이블의 사용자라는 뜻이다.

주석 `ON UPDATE CURRENT\_TIMESTAMP`는 행이 수정될 때 `updated\_at` 시간이 자동으로 현재 시간으로 바뀌게 하는 설정이다.

정리 `users`는 부모 테이블이고, `posts`, `comments`는 `users`의 `user\_id`를 참고하는 자식 테이블이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

샘플 데이터 입력

‘INSERT INTO’ 명령으로 유저, 게시물, 댓글 데이터를 여러 줄 입력했다.

```
INSERT INTO users (nickname, email) VALUES
('김민수', 'minsu@test.com'),
('이서연', 'seoyeon@test.com'),
('박지훈', 'jihoon@test.com'),
('최유진', 'yujin@test.com'),
('정하늘', 'haneul@test.com'),
('한도윤', 'doyun@test.com');
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

‘INSERT INTO users (nickname, email) VALUES ...’는 ‘users’ 테이블에 사용자 데이터를 추가하는 DML 명령이다.

확인

‘user_id’는 ‘AUTO_INCREMENT’라서 직접 넣지 않아도 1, 2, 3처럼 자동으로 생성된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

JOIN 연습 쿼리

sample.sql에는 테이블 생성과 데이터 입력뿐 아니라 JOIN 연습용 조회 쿼리도 들어 있다.

```
SELECT
  p.post_id,
  p.title,
  u.nickname,
  p.view_count,
  p.created_at
FROM posts p
JOIN users u
  ON p.user_id = u.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`JOIN users u ON p.user\_id = u.user\_id`는 게시물의 작성자 번호와 사용자 테이블의 번호를 연결해서 게시글과 작성자 닉네임을 함께 조회하는 부분이다.

```
SELECT
  p.post_id,
  p.title,
  COUNT(c.comment_id) AS comment_count
FROM posts p
LEFT JOIN comments c
  ON p.post_id = c.post_id
GROUP BY p.post_id, p.title
ORDER BY comment_count DESC
LIMIT 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`LEFT JOIN`은 댓글이 없는 게시글도 결과에 남길 수 있다. `COUNT`는 댓글 개수를 세고, `GROUP BY`는 게시글별로 묶어 통계를 낸다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

Workbench에서 데이터 수정

캡처에서는 `users` 테이블 결과 그리드에서 데이터를 확인하고, 직접 수정/추가한 상태가 보였다.

```
user_id | nickname | email | password | created_at
1 | 김민수 | minsu@test.com | 1234 | 2026-05-25 11:10:57
2 | 이서연 | seoyeon@test.com | 2345 | 2026-05-26 11:10:57
3 | 박지훈 | jihoon@test.com | 3456 | 2026-05-27 11:10:57
4 | 최유진 | yujin@test.com | 4567 | 2026-05-28 11:10:57
5 | 정하늘 | haneul@test.com | 6789 | 2026-05-29 11:10:57
6 | 한도윤 | doyun@test.com | 7890 | 2026-05-29 11:10:57
7 | 김동현 | sollee8404@gmail.com | 0629 | 2026-05-29 11:18:00
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

확인 sample.sql 원본의 `users` 테이블에는 `password` 컬럼이 없으므로, 캡처의 결과는 SQL 파일 실행 후 Workbench에서 컬럼을 추가하거나 기존 테이블 구조를 수정한 상태로 볼 수 있다.

주석 Result Grid에서 직접 값을 수정한 뒤 Apply를 누르면 실제 DB에 변경사항이 반영된다.

주의 비밀번호를 실제 서비스에서 `1234`, `0629`처럼 평문으로 저장하면 위험하다. 실무에서는 비밀번호를 해시 처리해서 저장한다.

정리 오늘 흐름은 `sample.sql` 실행 -> users/posts/comments 테이블 생성 -> 샘플 데이터 입력 -> JOIN 조회 연습 -> Workbench Result Grid에서 users 데이터 수정으로 정리할 수 있다.

MySQL SELECT, WHERE, JOIN 조회 정리

오늘은 `users` 테이블을 기준으로 SELECT 조회, WHERE 조건, LIKE 텍스트 검색, 날짜 조건, SQL 인젝션, 집계 함수, 정렬, 그룹화, JOIN까지 이어서 실습했다.

핵심 `SELECT`는 데이터를 조회하는 명령이고, `WHERE`는 조회할 행을 거르는 조건이다. 필요한 컬럼만 고르면 화면이 깔끔해지고, 조건을 붙이면 원하는 데이터만 추출할 수 있다.

기본 SELECT 조회

```
select * from users; -- users 테이블 전체조회
select nickname from users; -- users 테이블에서 닉네임 전체 조회
select user_id, nickname from users; -- users 테이블에서 아이디와 닉네임만 전체 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `\*`는 모든 컬럼을 뜻한다. `select nickname`처럼 컬럼명을 적으면 필요한 컬럼만 조회한다.

주석 `select user\_id, nickname`은 사용자 번호와 닉네임처럼 필요한 항목만 골라서 보는 방식이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

WHERE 조건 조회

```
select * from users where user_id = 3; -- user_id가 3인 사용자 한 명만 조회
select * from users where user_id = 3 OR user_id = 5; -- user_id가 3이거나 5인 사용자 조회
select * from users where user_id in(1,2,3); -- user_id가 1, 2, 3 중 하나인 사용자 조회
select * from users where user_id <= 3; -- user_id가 3 이하인 사용자 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `where user\_id = 3`은 `user\_id` 값이 정확히 3인 행만 가져온다.

주석 `OR`은 둘 중 하나라도 조건이 맞으면 조회한다. `user\_id = 3 OR user\_id = 5`는 3번 또는 5번 사용자를 찾는다.

주석 `IN`은 여러 값 중 하나와 일치하는지 검사한다. `user\_id in(1,2,3)`은 `user\_id = 1 OR user\_id = 2 OR user\_id = 3`과 비슷한 의미이다.

주석 `<=`는 작거나 같다는 뜻이다. 숫자 범위를 조건으로 걸 때 사용한다.

텍스트 검색

```
select * from users where nickname = '이서연'; -- 닉네임이 정확히 이서연인 사용자 조회
select * from users where nickname like '이서연'; -- 닉네임이 이서연 패턴과 일치하는 사용자 조회
select * from users where nickname like '김%'; -- 김으로 시작하는 닉네임 조회
select * from users where nickname like '%수'; -- 수로 끝나는 닉네임 조회
select * from users where nickname like '%서연%'; -- 서연이 포함된 닉네임 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `=`는 값이 정확히 같은지 비교한다.

주석 `LIKE`는 문자열 패턴 검색에 사용한다. `%`가 없으면 사실상 정확히 같은 값 검색처럼 동작한다.

주석 `김%`는 김으로 시작하고 뒤에는 어떤 글자가 와도 된다는 뜻이다.

주석 `%수`는 앞에는 어떤 글자가 와도 되고 마지막이 수로 끝나는 값을 찾는다.

주석 `%서연%`은 앞뒤에 어떤 글자가 있어도 중간에 서연이 들어가면 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

날짜 조건 조회

```
select * from users where created_at < '2026-05-28'; -- 2026-05-28 00:00:00 이전 데이터 추출
select * from users where created_at >= '2026-05-29' AND created_at < '2026-05-30'; -- 2026-05-29 하루 데이터만 추출
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의 `created\_at < '2026-05-28'`은 2026-05-28 하루 전체가 아니라 `2026-05-28 00:00:00`보다 이전 데이터를 뜻한다.

핵심 특정 날짜 하루만 가져올 때는 `created\_at >= '2026-05-29' AND created\_at < '2026-05-30'`처럼 시작일은 포함하고 다음날 시작 전까지 조회하는 방식이 안전하다.

이메일 검색 예제

```
-- email주소에 test가 들어있는 사람들 추출
select * from users where email like '%test%';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `%test%`는 이메일 앞뒤 어디든 `test` 문자열이 포함되어 있으면 조회한다.

로그인 조건 조회

```
-- 입력창: id, pw
select * from users where email = 'sollee8404@gmail.com' AND password = '0629';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 로그인은 보통 사용자가 입력한 id/email과 password가 모두 맞는지 확인한다.

주석 `AND`는 양쪽 조건이 모두 참이어야 한다. 여기서는 이메일도 맞고 비밀번호도 맞아야 조회 결과가 나온다.

주의 실제 서비스에서는 비밀번호를 평문으로 저장하지 않는다. 보통 bcrypt 같은 방식으로 해시 처리해서 저장한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

SQL 인젝션

SQL 인젝션은 사용자가 입력한 값이 SQL 문장 일부처럼 실행되어, 의도하지 않은 로그인 우회, 데이터 조회, 테이블 삭제 등이 발생하는 공격이다.

```
-- 비밀번호 조건을 주석 처리해서 강제 로그인처럼 만드는 예
select * from users where email = 'sollee8404@gmail.com'; -- ; AND password = '0629';

-- 뒤에 DROP TABLE을 붙여 테이블 삭제를 시도하는 예
select * from users where email = 'sollee8404@gmail.com'; DROP TABLE users; -- AND password = '0629';

-- 항상 참인 조건을 붙여 전체 사용자가 조회될 수 있는 예
select * from users where email = 'sollee8404@gmail.com' OR 1=1; -- ; AND password = '0629';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의 `OR 1=1`은 항상 참이다. 로그인 SQL에 이런 입력이 섞이면 비밀번호가 틀려도 조건 전체가 참이 되어버릴 수 있다.

주의 `--`는 SQL에서 주석으로 쓰인다. 공격자가 뒤쪽 조건을 주석 처리하면 원래 확인해야 할 비밀번호 조건이 무시될 수 있다.

SQL 인젝션 방어 방법

핵심 사용자 입력값을 SQL 문자열에 직접 이어붙이지 말고, 파라미터 바인딩 또는 ORM을 사용한다.

파라미터 바인딩	SQL 문장과 입력값을 분리해서 DB에 전달하는 방식
Prepared Statement	미리 SQL 구조를 정해두고 값만 나중에 안전하게 넣는 방식
ORM	객체와 테이블을 연결해서 SQL을 직접 조립하는 실수를 줄여주는 도구

주석 ORM은 Object Relational Mapping의 약자이다. Java의 JPA, Node.js의 Sequelize/Prisma, Python의 SQLAlchemy 같은 도구가 ORM에 해당한다.

정리 실무에서는 사용자가 입력한 이메일, 비밀번호, 검색어를 SQL 문자열에 직접 붙이지 않고 ORM이나 파라미터 바인딩으로 처리한다고 이해하면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

테이블 데이터 개수 구하기

```
select count(*) from users;
select count(*) from users where email like '%test%';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `count(\*)`는 조건에 맞는 행의 개수를 센다.

주석 첫 번째 쿼리는 전체 사용자 수를 구하고, 두 번째 쿼리는 이메일에 `test`가 들어간 사용자 수만 구한다.

정렬해서 가져오기

```
select * from users order by nickname;
select * from users where email like '%test%' order by nickname desc; -- 기본정렬 asc, 반대정렬 desc
select * from users where email like '%test%' order by nickname asc, email desc;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `order by nickname`은 닉네임 기준으로 정렬한다. 기본값은 오름차순 `ASC`이다.

주석 `DESC`는 내림차순 정렬이다.

주석 `order by nickname asc, email desc`는 먼저 닉네임을 오름차순으로 정렬하고, 닉네임이 같으면 이메일을 내림차순으로 정렬한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

중복 제거와 그룹화

```
select distinct(nickname) from users; -- 중복을 제거한 닉네임 목록 조회
select nickname, count(nickname) from users group by nickname; -- 닉네임별 데이터 개수 조회
select nickname, count(nickname), max(user_id) from users group by nickname; -- 닉네임별 개수와 가장 큰 user_id 조회
select nickname, count(nickname), max(user_id), min(user_id) from users group by nickname; -- 닉네임별 개수, 최대 user_id, 최소 user_id 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `distinct(nickname)`은 같은 닉네임이 여러 번 있어도 한 번만 보여준다.

주석 `group by nickname`은 같은 닉네임끼리 묶는다. 묶은 뒤 `count(nickname)`으로 닉네임별 개수를 셀 수 있다.

주석 `max(user\_id)`는 그룹 안에서 가장 큰 `user\_id`를 구한다.

주석 `min(user\_id)`는 그룹 안에서 가장 작은 `user\_id`를 구한다.

핵심 `GROUP BY`를 쓰면 행을 하나씩 보는 것이 아니라 같은 기준끼리 묶어서 통계를 낼 수 있다.

게시물 전체 조회

```
select * from posts;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `posts` 테이블의 모든 게시물 데이터를 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

게시물과 작성자 같이 조회하기: Sub Query

```
select
post_id,
(select nickname from users where users.user_id = p.user_id) AS user_name,
title, view_count, created_at
from posts AS p;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 서브쿼리는 SQL 안에 들어가는 또 다른 SELECT 문이다.

주석 `(select nickname from users where users.user\_id = p.user\_id)`는 게시물 한 줄마다 해당 작성자의 닉네임을 users 테이블에서 찾아온다.

주석 `AS user\_name`은 조회 결과 컬럼 이름을 `user\_name`으로 보여주기 위한 별칭이다.

주석 `from posts AS p`는 `posts` 테이블에 `p`라는 짧은 별칭을 붙인 것이다.

게시물과 작성자 같이 조회하기: JOIN

```
select p.post_id, u.nickname, p.title, p.view_count
from posts AS p
join users AS u ON p.user_id = u.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `JOIN`은 두 테이블을 동시에 조회하면서, `ON` 조건이 일치하는 데이터끼리 묶어 보여준다.

주석 `p.user\_id = u.user\_id`는 게시물 테이블의 작성자 번호와 사용자 테이블의 사용자 번호가 같은 행끼리 연결한다는 뜻이다.

핵심 작성자 닉네임처럼 다른 테이블에 있는 정보를 함께 보고 싶을 때 JOIN을 사용한다. 서브쿼리보다 가능하지만, 테이블 관계를 묶어서 조회할 때는 JOIN이 더 자주 쓰인다.

정리 오늘 SQL 조회 흐름은 `SELECT 기본 조회 -> WHERE 조건 -> LIKE 검색 -> 날짜 범위 -> 로그인 조건 -> SQL 인젝션 주의 -> COUNT/ORDER BY/GROUP BY -> 서브쿼리와 JOIN` 순서로 보면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

2026.06.04

데이터베이스 조회와 JOIN

핵심

오늘 내용은 `users`, `posts` 테이블을 기준으로 데이터 조회, 실행 계획 확인, subquery와 JOIN의 차이, LEFT/RIGHT JOIN으로 누락 데이터를 찾는 흐름이다.

확인

초반의 `show databases`, `use`, `show tables`, 기본 `select`는 기존 데이터베이스 복습 내용이므로 짧게 압축하고, 새로 중요한 부분인 `LIMIT`, `explain`, subquery 성능, JOIN 결과 차이를 중심으로 정리한다.

복습: 데이터베이스 선택과 기본 조회

```
show databases; -- 데이터베이스 목록 조회
use my_community; -- 사용할 기본 데이터베이스 선택

-- 테이블 목록 조회
show tables;

-- users 테이블 데이터 수 조회
select count(*) from users;

-- users 테이블 내용 조회
select * from users; -- users 테이블 조회 >> 너무 많으면 위험할수도있음
select * from users LIMIT 5; -- 5개 제한 조회
select * from users order by user_id desc LIMIT 5; -- user_id 역순 5개 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`select \* from users`는 모든 행과 모든 열을 가져오므로 데이터가 많으면 위험할 수 있다. 실습이나 확인용이면 `LIMIT`을 붙여서 일부만 조회하는 습관이 좋다.

주석

`order by user\_id desc LIMIT 5`는 `user\_id`가 큰 순서, 즉 최근에 만들어진 사용자부터 5개를 보는 형태로 사용할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

조건 조회와 인덱스 주의

```
select * from users where email = 'sollee8404@gmail.com' and password = '0629';
select * from users where password = '0629' and email = 'sollee8404@gmail.com'; -- 폴스캔, 이렇게짜면안됨 email은 unique라
인덱스값으로찾는데 password는 인덱스가 없어서 전체를 찾음
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `email`이 `unique`라면 DB가 인덱스를 이용해 빠르게 사용자를 찾을 수 있다. 반대로 `password`에는 보통 인덱스가 없기 때문에 password 조건을 중심으로 찾으면 전체 데이터를 훑는 폴스캔 위험이 있다.

주의 실제 서비스에서는 비밀번호를 평문으로 조회/저장하면 안 된다. 수업 SQL은 조건 조회와 인덱스 설명용 예시로 보고, 운영에서는 해시된 비밀번호를 비교해야 한다.

실행 계획 확인

```
-- 실행 계획 확인
explain select * from users;
explain select * from users where user_id = 3;
explain select * from users where user_id > 5;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `explain`은 DB가 SQL을 어떻게 실행할지 보여준다. 인덱스를 타는지, 전체 테이블을 훑는지 확인할 때 사용한다.

주석 `user\_id = 3`처럼 기본키/인덱스 조건이면 특정 행을 빠르게 찾을 수 있고, `user\_id > 5`는 범위 조건으로 여러 행을 탐색한다.

게시물 목록 조회

```
-- 게시물 목록을 출력 (작성자(닉네임), 제목, 조회수, 등록일자)
select * from posts;
select user_id, title, view_count, created_at from posts;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 게시물 목록 화면에서는 모든 컬럼이 아니라 필요한 컬럼만 가져오는 것이 좋다. 예시는 작성자 식별값, 제목, 조회수, 등록일자를 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

Subquery로 작성자 닉네임 가져오기

```
-- subquery
-- 장점: 단순하다
-- 단점: posts 데이터 수만큼 서브쿼리가 실행됨 >> 성능하락

select
(select nickname from users u where u.user_id = p.user_id) AS nink,
title,
view_count,
created_at
from
posts AS p; -- 테이블 뒤에 AS생략가능 users u >> users AS u
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 subquery는 SQL 구조가 단순해서 이해하기 쉽다. 하지만 게시물마다 내부 쿼리가 반복 실행될 수 있어 데이터가 많아지면 성능이 떨어질 수 있다.

주석 `posts AS p`, `users u`처럼 테이블에 별칭을 붙이면 SQL을 짧게 쓸 수 있다. `AS`는 생략할 수 있다.

JOIN으로 users와 posts 연결

```
-- JOIN (inner join = 교집합) -- on뒤에는 교집합 조건
select * from posts JOIN users on posts.user_id = users.user_id;
select * from posts INNER JOIN users on posts.user_id = users.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `INNER JOIN`은 양쪽 테이블에 모두 매칭되는 데이터만 가져온다. 즉 게시물의 `user\_id`와 사용자 테이블의 `user\_id`가 일치하는 행만 결과에 나온다.

주석 `JOIN`만 써도 보통 `INNER JOIN`과 같은 의미로 동작한다.

LEFT JOIN / RIGHT JOIN

```
-- 탈퇴하고 없는 사람 글도 조회
select * from posts LEFT JOIN users on posts.user_id = users.user_id; -- 설명추가
select * from posts RIGHT JOIN users on posts.user_id = users.user_id; -- 설명추가
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `LEFT JOIN`은 왼쪽 테이블인 `posts`를 기준으로 전부 보여주고, 매칭되는 사용자가 있으면 붙인다. 사용자가 삭제되어도 글을 기준으로 확인할 수 있다.

주석 `RIGHT JOIN`은 오른쪽 테이블인 `users`를 기준으로 전부 보여주고, 매칭되는 게시물이 있으면 붙인다. 게시물을 작성하지 않은 사용자도 확인할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

탈퇴한 사람 글만 찾기

```
-- 탈퇴한 사람 글만 조회
select * from posts LEFT JOIN users on posts.user_id = users.user_id
WHERE users.user_id is null;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `posts`에는 글이 남아 있는데 `users`에서 매칭되는 사용자가 없으면 `users.user\_id`가 `null`이 된다. 이 조건으로 탈퇴한 사용자의 글만 찾을 수 있다.

게시물 작성 내역이 없는 사람 찾기

```
-- 게시물 작성 내역이 없는사람
select * from posts RIGHT JOIN users on posts.user_id = users.user_id
WHERE posts.post_id is null;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `users`에는 사용자가 있는데 `posts`에서 매칭되는 게시물이 없으면 `posts.post\_id`가 `null`이 된다. 이 조건으로 글을 작성하지 않은 사용자를 찾을 수 있다.

JOIN 그림 요약

INNER JOIN	A와 B의 교집합만 조회
LEFT JOIN	왼쪽 A는 전부 조회하고, B는 매칭되는 것만 붙임
RIGHT JOIN	오른쪽 B는 전부 조회하고, A는 매칭되는 것만 붙임

- LEFT JOIN + `WHERE B.key IS NULL`: A에는 있지만 B에는 없는 데이터
- RIGHT JOIN + `WHERE A.key IS NULL`: B에는 있지만 A에는 없는 데이터

FULL OUTER JOIN	A와 B 전체를 합쳐서 조회한다. MySQL에서는 기본 문법으로 직접 지원하지 않아 `LEFT JOIN`과 `RIGHT JOIN`을 `UNION`으로 조합하는 방식이 필요할 수 있다.
------------------------	--

정리 게시글과 사용자처럼 서로 연결된 테이블을 다룰 때는 subquery보다 JOIN을 우선 고려한다. JOIN은 어떤 테이블을 기준으로 삼는지, 매칭되지 않는 데이터를 남길지 버릴지를 기준으로 선택한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

데이터베이스 복습: CRUD와 게시판 구조

핵심

데이터베이스를 배우는 이유는 게시판 같은 서비스를 만들기 위해서다. 게시판은 글/댓글 데이터를 저장하고, 사용자를 인증하고, 서버와 통신하며, 필요하면 실시간 기능까지 붙는다.

CRUD	생성(Create), 조회(Read), 갱신(Update), 삭제>Delete)
게시판	데이터 CRUD + 인증 + 통신(REST API) + 웹소켓
웹소켓	채팅처럼 실시간으로 주고받는 기능에 사용

주석

게시판을 만들 때 DB는 글, 댓글, 사용자, 조회수, 작성일 같은 데이터를 저장하고 꺼내는 핵심 역할을 한다.

조회: SELECT

```
SELECT 컬럼명1, 컬럼명2
FROM 테이블명
WHERE 필터링조건;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

수업 메모의 'SELCT'는 'SELECT' 오타로 정리했다. 'SELECT'는 원하는 컬럼을 조회하고, 'WHERE'는 필요한 행만 필터링한다.

주석

게시물과 댓글을 한 번에 보여주려면 서로 다른 테이블을 연결해야 하므로 'JOIN'을 사용한다.

생성, 갱신, 삭제

```
-- 생성(등록)
INSERT INTO 테이블명 (컬럼1, 컬럼2)
VALUES (값1, 값2);

-- 갱신(업데이트)
UPDATE 테이블명
SET 컬럼명 = 새값
WHERE 조건;

-- 삭제
DELETE FROM 테이블명
WHERE 조건;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

'UPDATE'와 'DELETE'는 'WHERE' 없이 실행하면 테이블 전체가 바뀌거나 삭제될 수 있다. 실습할 때도 먼저 'SELECT ... WHERE ...'로 대상이 맞는지 확인하는 습관이 좋다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

데이터베이스 Index 개념

인덱스는 책의 색인처럼 특정 컬럼 값을 빠르게 찾기 위한 별도 자료구조다. 예를 들어 `users.email`에 인덱스가 있으면 DB가 모든 사용자를 처음부터 끝까지 훑지 않고, 인덱스를 통해 해당 이메일 위치를 빠르게 찾을 수 있다.

장점	`WHERE`, `JOIN`, `ORDER BY`에서 검색 속도가 빨라질 수 있음
단점	인덱스도 저장 공간을 차지함
단점	`INSERT`, `UPDATE`, `DELETE` 때 인덱스도 같이 수정해야 하므로 쓰기 성능이 느려질 수 있음
기준	자주 검색하거나 JOIN 조건에 쓰는 컬럼에 인덱스를 만든다
주의	값 종류가 너무 적은 컬럼은 인덱스 효율이 낮을 수 있다

-- 예시: email로 자주 사용자를 찾는 경우

```
CREATE INDEX idx_users_email ON users(email);
```

-- 예시: posts와 users를 user_id로 자주 JOIN하는 경우

```
CREATE INDEX idx_posts_user_id ON posts(user_id);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `email`처럼 유일하거나 선택도가 높은 컬럼은 인덱스 효율이 좋다. 반대로 성별처럼 값 종류가 적은 컬럼은 인덱스를 만들어도 효과가 작을 수 있다.

인덱스를 두 개 만들면?

인덱스를 두 개 만든다는 것은 DB가 검색할 때 사용할 수 있는 길을 두 개 만들어 두는 것이다. 예를 들어 `email` 인덱스와 `created\_at` 인덱스가 있으면, 이메일 검색에는 `email` 인덱스를 쓰고 최신순 정렬/범위 조회에는 `created\_at` 인덱스를 쓸 수 있다.

```
CREATE INDEX idx_users_email ON users(email);
```

```
CREATE INDEX idx_users_created_at ON users(created_at);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 인덱스가 여러 개 있다고 해서 한 쿼리에서 항상 전부 쓰는 것은 아니다. DB 옵티마이저가 실행 계획을 보고 가장 유리한 인덱스를 고른다.

주의 인덱스를 많이 만들수록 조회 선택지는 늘지만, 데이터 등록/수정/삭제 때 관리해야 할 인덱스도 늘어난다. 그래서 "많을수록 좋다"가 아니라 "자주 쓰는 조회 조건에 맞게 필요한 만큼" 만드는 것이 중요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - 데이터베이스 MySQL 개념정리	date

복합 인덱스

두 컬럼을 따로 인덱스로 만드는 것과, 두 컬럼을 묶어서 하나의 복합 인덱스로 만드는 것은 다르다.

```
-- 각각 따로 만드는 인덱스
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_password ON users(password);

-- 두 컬럼을 묶은 복합 인덱스
CREATE INDEX idx_users_email_password ON users(email, password);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `(email, password)` 복합 인덱스는 `email` 조건부터 사용하는 쿼리에 유리하다. 인덱스는 앞 컬럼부터 정렬된 구조라서 컬럼 순서가 중요하다.

주의 실제 서비스에서 `password`를 인덱싱하거나 평문으로 비교하는 방식은 좋지 않다. 수업에서는 인덱스 동작을 이해하기 위한 예시로 보고, 운영에서는 비밀번호 해시와 인증 로직을 사용한다.

MySQL Workbench 조회 제한 설정

```
Edit > Preferences... > SQL Editor > SQL Execution > Limit Rows Count
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Workbench에서 조회 결과 행 수 제한을 설정할 수 있다. 실수로 `SELECT \* FROM users;`처럼 많은 데이터를 조회했을 때 너무 많은 행을 가져오는 것을 줄이는 안전장치다.

정리 게시판 DB를 다룰 때는 CRUD 문법을 기본으로 알고, 조회가 많아지는 컬럼에는 인덱스를 고려한다. 다만 인덱스는 조회를 빠르게 하는 대신 쓰기 비용과 저장 공간을 늘리므로, `EXPLAIN`으로 실제 실행 계획을 확인하면서 설계해야 한다.